DEEP LEARNING & REINFORCEMENT LEARNING · MINI PROJECT REPORT

# Snake × Q-Learning

A snake that teaches itself to play, using Reinforcement Learning.


TEAM

- Evans Pereira — Roll no. 10689
- Dakshat Jain — Roll no. 10663
- Shaurya Sanil — Roll no. 10698


LINKS

**GitHub repository**
github.com/DraXx-Van/RL

**Video 2, with voiceover (3 min 50 s)**
github.com/DraXx-Van/RL/blob/main/demo_video/Video_2_SFX_Voiceover.mp4

**Video 1, sound effects only (3 min 50 s)**
github.com/DraXx-Van/RL/blob/main/demo_video/Video_1_SFX.mp4

## 1 Topic and goal

Our topic is **Reinforcement Learning**. We built the classic Snake game and, instead of programming how to play it, let the snake **learn by trial and reward**. It plays thousands of games, earns points for eating apples and penalties for crashing, and gradually works out which move is best in every situation.

The goal was to understand Q-learning properly and to make the learning **visible**: you can watch the table of scores change while the snake plays.

## 2 What we did

- **Built the game as a learning environment**: a 15 × 15 Snake world that gives a reward after every move (+10 apple, −10 crash).
- **Designed what the snake sees**: just 7 yes/no facts (danger ahead / right / left, and where the apple is), which gives 128 situations. We tested three designs; the compact one that describes the apple relative to the snake's heading worked as well as a larger 11-fact version.
- **Implemented the Q-learning agent**: a table of 128 situations × 3 moves, an exploration rule (random moves early, learned moves later), and the update formula.
- **Trained and tested it**: 3,000 games (about 10 seconds), then 100 fresh test games with learning switched off.
- **Built an interactive app** (Pygame): train it, watch the trained snake, or play yourself, with the Q-table updating live next to the game, and sound effects.
- **Checked our work**: 26 automated tests, plus scripts to evaluate the snake, read the table in plain English, and export all 128 situations.
- **Made an explainer video** (3 min 50 s) that walks through the whole idea with real numbers from our own training run.

| 128 × 3 | 3,000 | 23.2 | 26 |
|---|---|---|---|
| situations × moves in the Q-table | training games, about 10 seconds | average apples (a random snake: 0.12) | automated tests, all passing |

## 3 How the snake learns

**SITUATION**

7 yes/no facts from the snake's own point of view: danger ahead / right / left, apple ahead / behind / left / right. $2^7$ = **128 situations**.

**MOVES**

3 possible moves: **straight, turn right, turn left**. These are the 3 columns of the table.

**REWARDS**

**+10** for eating an apple, **−10** for crashing (wall, itself, or wandering too long), **0** otherwise.

The **Q-table** has one row per situation and one number per move: how good that move is in that situation, *in the long run*. After every move the snake corrects one number:

$$Q[s][a] \leftarrow Q[s][a] + \alpha \times (\text{reward} + \gamma \times \text{best score in the next situation} - Q[s][a])$$

- **α = 0.1**: how big each correction is.
- **γ = 0.9**: how much the future counts. This is how the value of an apple travels backward to the moves that led to it.
- **Exploration**: 100% random moves at first, shrinking by 0.5% after every game, down to 1%.

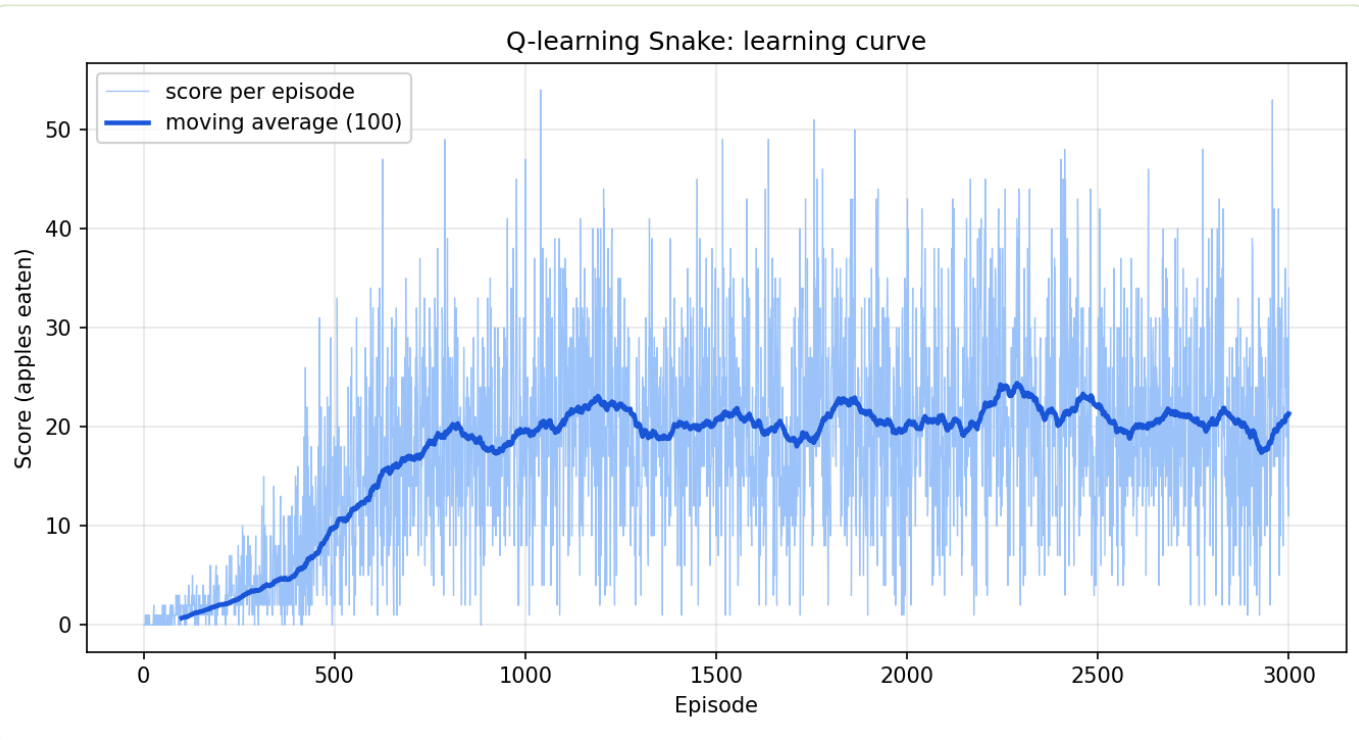
**A REAL EXAMPLE FROM OUR TRAINING RUN (GAME #5)**

The snake, with an empty table, moves forward and **eats an apple**. Reward = +10. It knows nothing yet about the next situation, so the target is 10 + 0.9 × 0 = **10**. New score = 0 + 0.1 × (10 − 0) = **1.0**. That cell goes from 0.0 to 1.0. After thousands of games, good moves grow to about +10.

## 4 Results

| Agent | Avg. apples | Best |
|---|---|---|
| Random snake | 0.12 | 2 |
| **Trained Q-agent** | **23.2** | **45** |

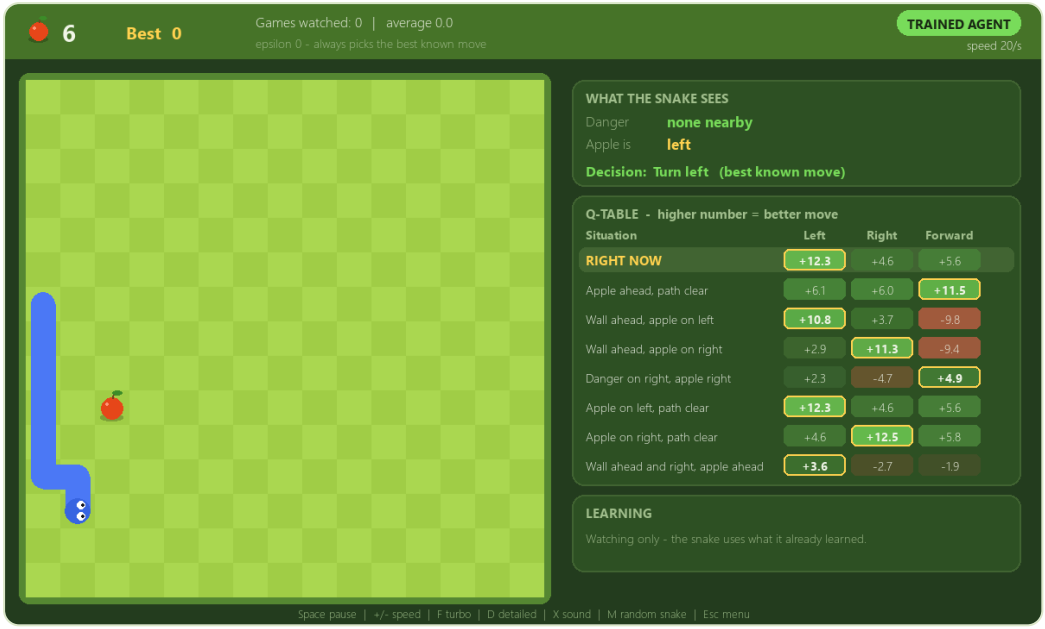
Average over **100 fresh games** the snake never trained on, with learning switched off. Training took about 10 seconds.




The average score rises from about 0 to about 20 as the snake learns.

## 5 The app

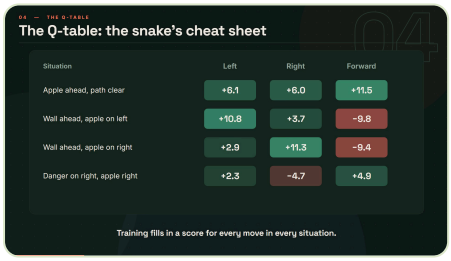
The app shows the game on the left and, on the right, what the snake sees, the part of the Q-table it is looking at, and what it learned from the last move. You can train the snake from an empty table, watch the trained snake, or play yourself.



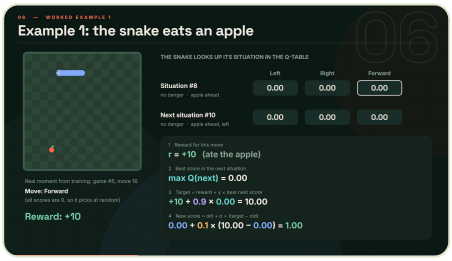

The trained snake playing, with its Q-table beside it.

## 6 The explainer video

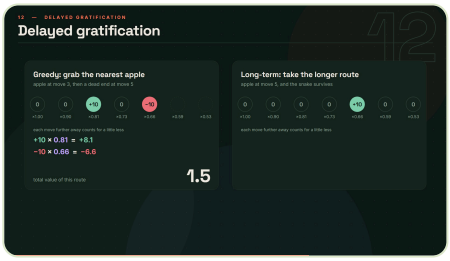
We made a **3 min 50 s video** to explain the project visually. It covers what the snake sees, how the Q-table works, the update formula with three **real examples taken from our training run** (eating an apple, crashing, and a plain move), why the formula looks at the next situation, what a Q-value really is, delayed gratification, and the results.




The Q-table




A real update from training




Delayed gratification

There are two versions of the video: **Video 2** with a short voiceover, and **Video 1** with sound effects only, for presenting live while speaking over it.
Video 2: github.com/DraXx-Van/RL/blob/main/demo_video/Video_2_SFX_Voiceover.mp4
Video 1: github.com/DraXx-Van/RL/blob/main/demo_video/Video_1_SFX.mp4
*GitHub cannot play large files in the page: open the link and press "Download raw file" (about 21 MB each), then play it in any video player.*

## 7 Limitations and next steps

The snake only knows the 7 facts above. It cannot see the shape of its own body, so in long games it can trap itself, and it cannot plan long detours. A table is also too small for bigger problems. The natural next step is a **DQN (Deep Q-Network)**: replace the table with a small neural network so the snake can use a much richer view of the board.

## 8 Tools used

Python, NumPy, Pygame (game and app), Matplotlib (graphs), pytest (26 tests). The video was built from code with Remotion, using the real numbers from our training run.


Evans Pereira · Dakshat Jain · Shaurya Sanil